Assignment 03 Report

Course

CSE323: Advanced Embedded Systems Design
CSE411/CSE411s: Real-Time and Embedded Systems Design

Assignment

Assignment 03: Interrupt Interaction with a FreeRTOS Task

Objective

The purpose of this assignment is to demonstrate how a hardware interrupt interacts with a real-time operating system. The implemented solution uses a GPIO interrupt on Port F to notify a FreeRTOS task. The ISR performs only the minimum required work, while the actual application response is handled inside a task.

Selected Interrupt Type

The selected interrupt source is the GPIO Port F interrupt generated by SW1 on PF4.

System Design

The application contains three main parts:

1. GPIO interrupt configuration on PF4.
2. An ISR named `GPIOF_Handler()`.
3. A FreeRTOS task named `vActionTask()` that waits for a signal from the ISR.

A second task named `vHeartbeatTask()` is also used to show that the scheduler is alive during normal execution.

Implementation Summary

1. GPIO Configuration

Port F was configured so that:

- PF4 works as a digital input with internal pull-up resistance.
- PF1, PF2, and PF3 work as output pins for LED indication.
- PF4 generates an interrupt on the falling edge, which corresponds to pressing SW1.

2. ISR Behavior

The interrupt service routine `GPIOF_Handler()` performs the following steps:

- clears the interrupt flag for PF4
- increments `g_interruptCount`
- toggles the blue LED indicator

- sends a direct task notification to the FreeRTOS action task using `vTaskNotifyGiveFromISR()`
- requests an immediate context switch using `portYIELD_FROM_ISR()` if a higher-priority task was woken

This follows the assignment requirement of keeping the ISR lightweight and avoiding heavy processing inside the interrupt context.

3. FreeRTOS Task Behavior

The action task `vActionTask()` waits indefinitely using `ulTaskNotifyTake(pdTRUE, portMAX_DELAY)`. When the interrupt notification arrives, the task:

- increments `g_taskWakeCount`
- records the tick count in `g_lastNotificationTick`
- toggles the red LED

This confirms that the actual application behavior is handled in task context rather than inside the ISR.

Debug and Verification Method

The project was built successfully in Keil without errors or warnings. After entering debug mode in the simulator:

- a breakpoint was placed inside `GPIOF_Handler()` at the line that increments `g_interruptCount`
- a second breakpoint was placed inside `vActionTask()` at the line that increments `g_taskWakeCount`
- the Watch window was used to monitor the following variables:
  - `g_interruptCount`
  - `g_taskWakeCount`
  - `g_lastNotificationTick`
  - `g_actionTaskHandle`

The SW1 button in the TM4C123 LaunchPad simulator was then pressed to generate the interrupt.

Experimental Results

The observed execution sequence was correct:

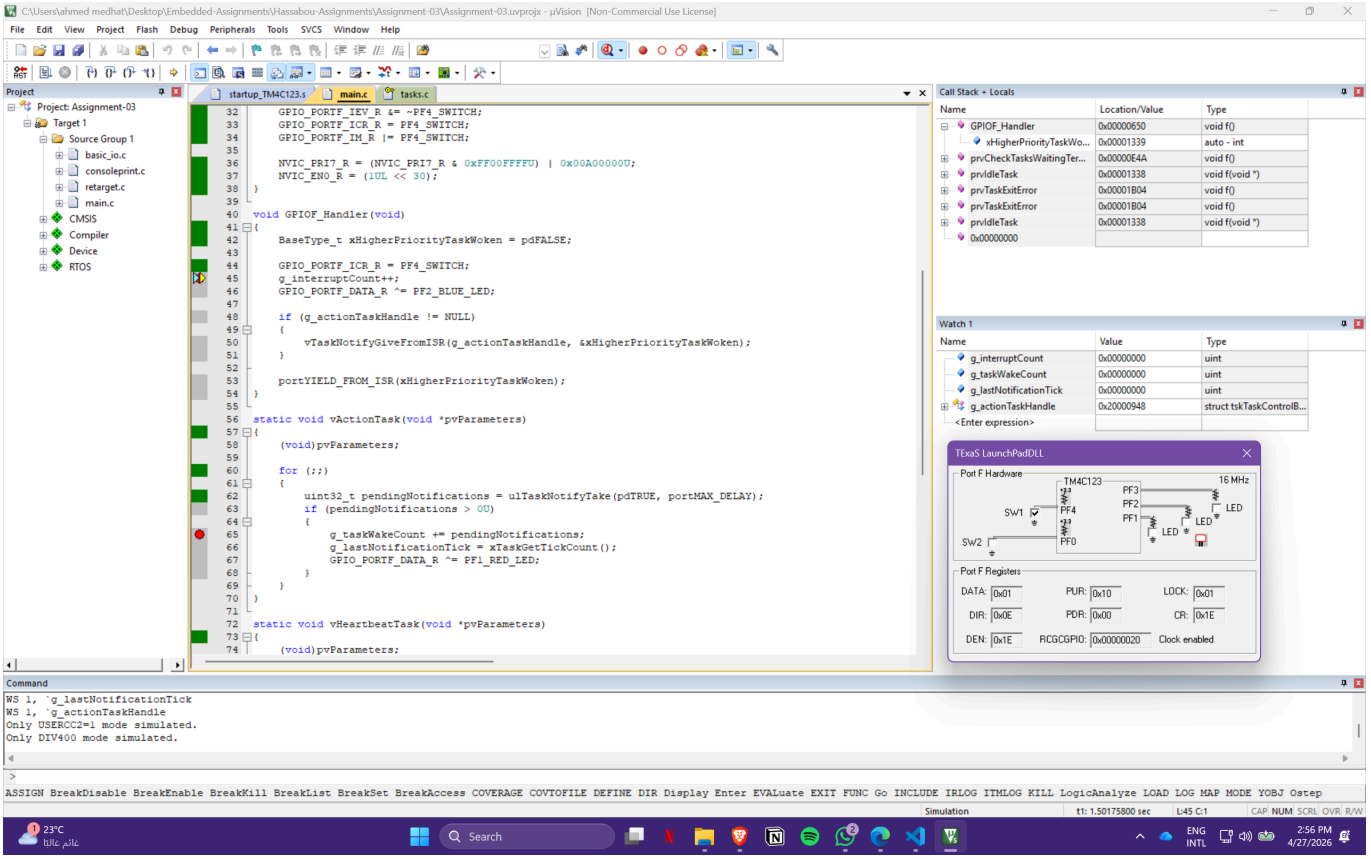
1. The first breakpoint was hit inside `GPIOF_Handler()`, proving that the GPIO interrupt was generated and the ISR was entered.
2. After stepping over the increment statement, `g_interruptCount` changed from `0` to `1`.
3. Execution was continued, and the second breakpoint was hit inside `vActionTask()`.
4. After stepping over the task-side counter update, `g_taskWakeCount` changed from `0` to `1`.
5. `g_lastNotificationTick` also changed, proving that the task resumed execution after being notified by the ISR.

These observations confirm correct synchronization between the hardware interrupt and the FreeRTOS task.

Screenshot Evidence

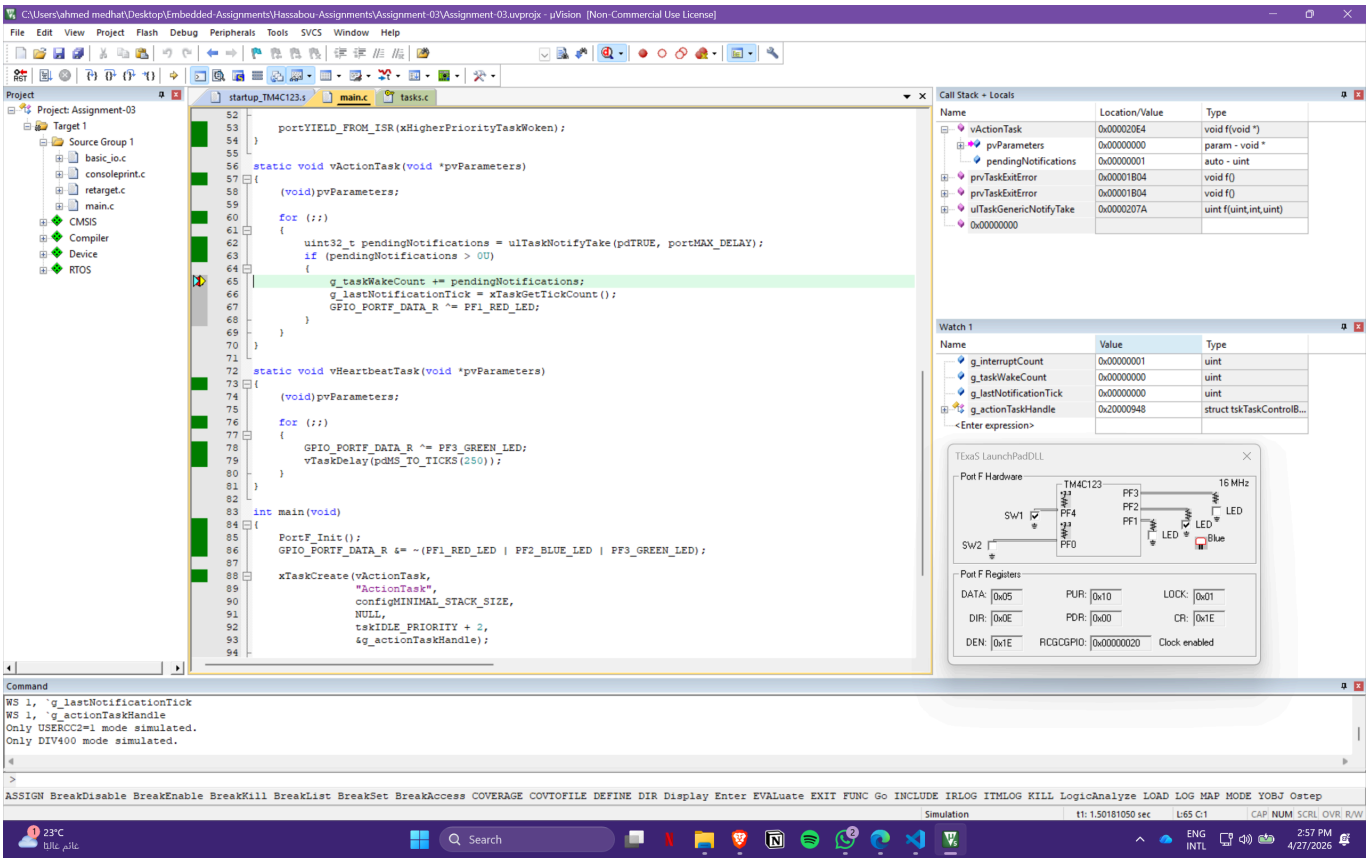
Screenshot 1: GPIO interrupt entered before ISR counter update

This screenshot shows execution stopped inside `GPIOF_Handler()` before the ISR counter increment line is completed. This proves that the processor entered the interrupt routine after SW1/PF4 was triggered.



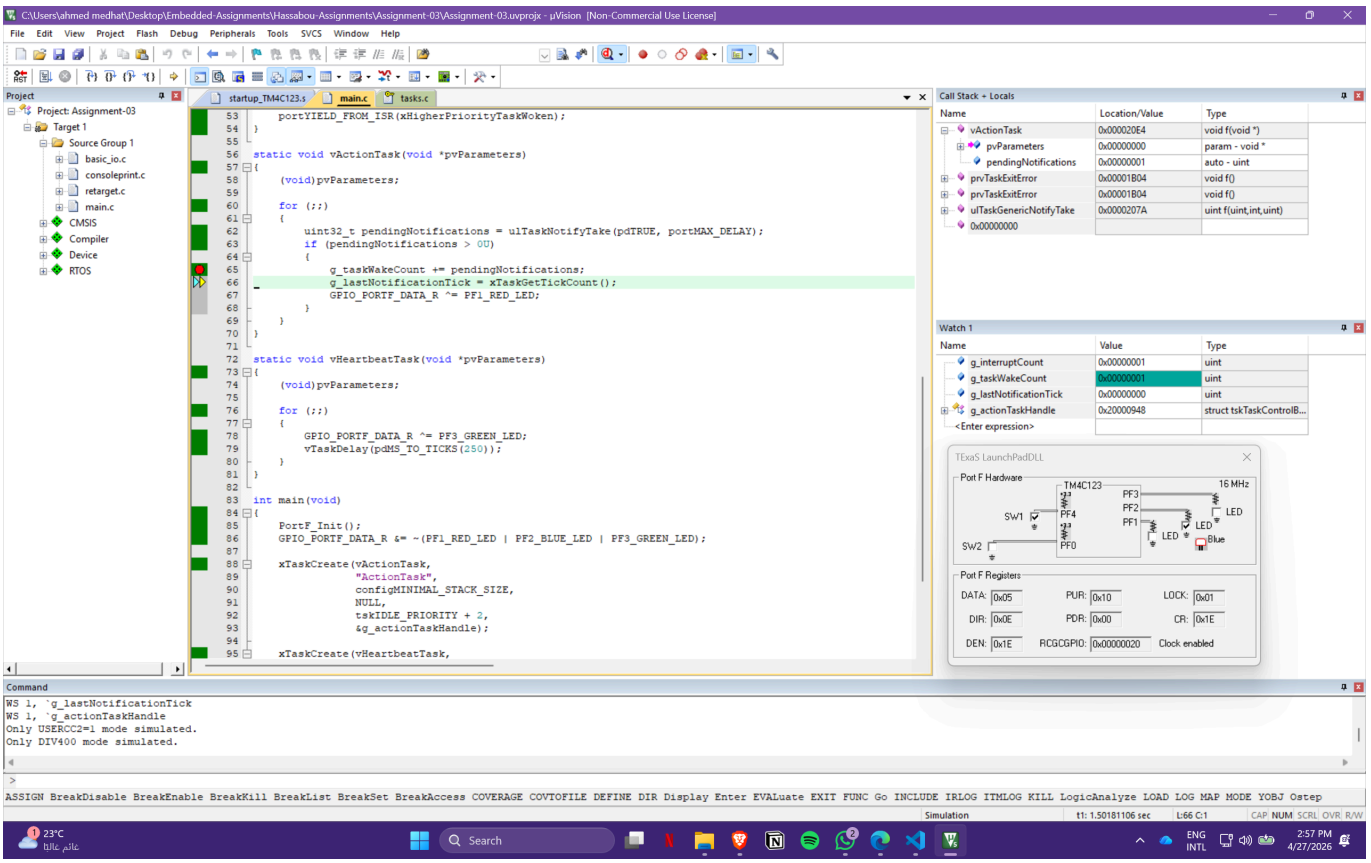
Screenshot 2: FreeRTOS task woke after ISR notification

This screenshot shows execution stopped inside `vActionTask()` after the interrupt notification caused the blocked task to resume. This proves the ISR successfully communicated with the task.



Screenshot 3: Interrupt-to-task synchronization verified

This screenshot shows the updated watch variables after stepping through the handler and the task. The values confirm that the interrupt count and task wake count both increased as expected.



Challenges Faced

The main challenge was verifying the exact sequence of execution between the ISR and the task. This was handled by using breakpoints and watch variables in Keil debug mode. Monitoring the variables directly made it possible to prove that the interrupt occurred first and the task resumed afterward.

Conclusion

The assignment objective was achieved successfully. A GPIO interrupt on PF4 was configured and linked to `GPIOF_Handler()`. The ISR used a FreeRTOS direct-to-task notification to wake `vActionTask()`, and the observed debugger evidence confirmed correct synchronization between interrupt context and task context. The implementation satisfies the assignment requirement of using an interrupt-driven mechanism to trigger a FreeRTOS task while keeping the ISR minimal.